

Week 4: Model Free Control

Mayuresh Thengzode 25B1816

1 Part 1: Algorithmic Derivations

Model-free control shifts focus from evaluating state values ($V(s)$) to action values ($Q(s, a)$) because the agent does not possess the transition probability dynamics ($P_{ss'}^a$) required to extract a policy from $V(s)$.

1.1 SARSA (On-Policy TD Control)

SARSA evaluates and improves the *same* policy that it uses to make decisions. The name reflects the experience tuple used in the update: $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$.

Derivation

We begin with the Bellman expectation equation for the action-value function under policy π :

$$Q^\pi(s, a) = R_s^a + \gamma \sum_{s' \in \mathcal{S}} P_{ss'}^a \sum_{a' \in \mathcal{A}} \pi(a'|s') Q^\pi(s', a')$$

In a model-free setting, we cannot compute the expectation over transitions ($\sum P_{ss'}^a$) or the policy ($\sum \pi$). Instead, TD learning substitutes the true expectation with an empirical sample target from the trajectory.

Replacing the expected return with the immediate reward plus the discounted value of the *actual* next state-action pair chosen by the policy gives the SARSA target:

$$\text{Target}_{\text{SARSA}} = R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$$

We then update our current estimate toward this target using a learning rate α :

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha (R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t))$$

1.2 Q-Learning (Off-Policy TD Control)

Q-learning optimizes a **target policy** (usually a fully greedy policy) while the agent behaves according to a separate **behavior policy** (usually an exploratory policy like ϵ -greedy).

Derivation

We begin with the Bellman optimality equation for action values:

$$Q^*(s, a) = R_s^a + \gamma \sum_{s' \in \mathcal{S}} P_{ss'}^a \max_{a' \in \mathcal{A}} Q^*(s', a')$$

To approximate this without the model transitions ($\sum P_{ss'}^a$), we sample the immediate reward R_{t+1} and the next state S_{t+1} by taking an exploratory action from our behavior policy.

However, instead of looking at the action the behavior policy actually selects next, we *bootstrap* assuming we will act perfectly optimally from that point onward by using $\max_{a'} Q(S_{t+1}, a')$.

This yields the Q-learning target:

$$\text{Target}_{\text{Q-learning}} = R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a')$$

Updating the estimate toward this optimal target gives:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left(R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t) \right)$$

2 Part 2: Structural Comparison

Table 1: Comparison of SARSA and Q-Learning

Feature	SARSA	Q-Learning
Core Nature	On-Policy: Learns the value of the policy it executes (including exploration errors).	Off-Policy: Learns the value of the optimal greedy policy while behaving exploratorily.
Target Update	$R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$	$R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a')$
Exploration Risk	Safe/Conservative: If exploration steps lead to negative penalties, SARSA adjusts paths to avoid them.	Aggressive/Optimistic: Evaluates paths based on perfect execution, ignoring exploration slips.
Convergence	Converges to π^* if policy becomes greedy over time (GLIE conditions) and $\sum \alpha_t = \infty, \sum \alpha_t^2 < \infty$.	Converges directly to Q^* independent of behavior policy, provided all state-action pairs are visited infinitely often.